FILE SYSTEMS
AND
FILE ORGANIZATION

A Comprehensive Study Guide

Concept of Files · File Organization & Accessing Techniques
(Indexed, Sequential, Hashed) · File Handling Functions
(Sorting, Merging, Indexing, Updating)

Prepared for University Examinations, Competitive Examinations & Quick Revision

TABLE OF CONTENTS

1 Concept of File 3

- 1.1 Definition of a File 3
- 1.2 Characteristics of a File 3
- 1.3 Types of Files 3
- 1.4 File Attributes 3
- 1.5 Need for Files 4
- 1.6 File Operations 4
- 1.7 Advantages and Limitations of File Systems 4
- 1.8 Real-World Examples 4
- 1.9 Exam Notes 4

2 File Organization and Accessing Techniques 5

- 2.A Indexed File Organization 5
- 2.B Sequential (Line Sequential) File Organization 6
- 2.C Hashed File Organization 8
- 2.D Comparison of the Three Techniques 10

3 File Handling Functions 11

- 3.A Sorting 11
- 3.B Merging 12
- 3.C Indexing 13
- 3.D Updating 15

CHAPTER 1: CONCEPT OF FILE

1.1 Definition of a File

A **file** is a named collection of related information (records) that is recorded on secondary storage such as a hard disk, SSD, or magnetic tape. It is the basic unit of storage that allows a computer to distinguish one set of data from another. A file may contain data, programs, text, images, or any sequence of bytes that has meaning to the user or the application that created it.

> **Formal Definition**
>
> A file is an organized collection of **related records** (or bytes) stored under a single name on a storage medium, which can be created, read, updated, and deleted as a single unit.

1.2 Characteristics of a File

- **Name:** Every file has a unique name within its directory, used to identify and access it.
- **Persistence:** Data in a file remains stored even after the program that created it terminates, and even after power is switched off (non-volatile storage).
- **Logical collection:** A file groups together logically related data, e.g., all employee records of a company.
- **Structured records:** Files are usually composed of records, and records are composed of fields.
- **Size:** A file has a measurable size in bytes, which may grow or shrink as records are added or removed.
- **Type:** A file has a type (text, binary, executable, image, etc.) that determines how it is interpreted.
- **Access permissions:** A file has associated permissions controlling who can read, write, or execute it.

1.3 Types of Files

Basis of Classification	Types	Description
Based on Content	Text Files	Store data in human-readable ASCII/Unicode form, e.g., .txt, .csv
	Binary Files	Store data in binary (0s and 1s) form not directly readable, e.g., .exe, .jpg
Based on Purpose	Master File	Contains relatively permanent data, e.g., employee master file
	Transaction File	Contains records of day-to-day transactions used to update the master file
	Report File	Contains formatted data meant for output/printing
Based on Organization	Sequential File	Records stored and accessed in a specific linear order
	Indexed File	Records accessed using an index pointing to their location
	Hashed (Direct) File	Records accessed directly using a hash function on the key

1.4 File Attributes

File attributes are **metadata** — properties associated with a file that describe it but are not part of the actual data content.

Attribute	Meaning
Name	Symbolic name used to identify the file, kept in human-readable form
Identifier (File ID)	A unique tag/number identifying the file within the file system
Type	Needed for systems that support different types, e.g., .txt, .exe, .dat
Location	Pointer to the device and the address of the file on that device
Size	Current size of the file in bytes, words, or blocks; also possibly maximum allowed size
Protection	Access-control information determining who can read, write, execute
Time, Date and User Identification	Data for creation, last modification, and last use; useful for security and monitoring

1.5 Need for Files

- To store large volumes of data permanently, beyond the lifetime of a single program execution.
- To allow data to be shared among multiple programs and users.
- To organize data logically so that it can be efficiently searched, updated, and retrieved.
- To provide a structured mechanism for backup, recovery, and security of data.

- To reduce data redundancy by maintaining a single common store of related information.

1.6 File Operations

The operating system provides system calls for performing basic operations on files. The six basic operations are:

Operation	Description
Create	Allocates space for a new file in the file system and makes an entry for it in the directory
Open	Prepares the file for reading/writing by loading its attributes and pointers into memory
Read	Transfers data from the file into memory, usually advancing a read pointer
Write	Transfers data from memory into the file, usually advancing a write pointer
Seek (Reposition)	Repositions the file pointer to a specific location for direct/random access
Close	Releases all resources/buffers associated with the open file and updates directory information
Delete	Removes the file's directory entry and frees the storage space it occupied
Truncate	Erases the contents of the file but keeps its attributes, resetting length to zero
Append	Adds new data to the end of an existing file
Rename	Changes the name of an existing file

1.7 Advantages and Limitations of File Systems

Advantages	Limitations
Provides permanent, non-volatile storage of data	Data redundancy may occur across multiple files
Simple to design and implement for small applications	Lack of data integrity constraints across files
Direct and fast access to data when organization is appropriate	Difficult to share data consistently between programs
Supports a hierarchical directory structure for organization	Limited concurrent access control compared to database systems
Provides basic security through access permissions	Searching unstructured/large files can be slow
Low overhead; does not require a DBMS	No built-in support for complex queries

1.8 Real-World Examples

- **Payroll System:** An "Employee Master File" stores permanent employee details (ID, name, salary), while a "Transaction File" stores monthly attendance to update salaries.
- **Library Management:** A "Books File" with fields for book ID, title, author, and availability status.
- **Banking System:** An "Accounts File" indexed by account number for instant balance lookup.
- **Telephone Directory:** A sequential file of subscriber records sorted by name for printing directories.
- **Student Database:** A hashed file of roll numbers used for instant retrieval of marks during result processing.

EXAM-ORIENTED NOTES — Chapter 1

- A file = collection of related records; a record = collection of related fields.
- Remember the 7 file attributes: Name, Identifier, Type, Location, Size, Protection, Time/Date/User.
- Six basic file operations: Create, Open, Read, Write, Seek/Reposition, Close (plus Delete, Truncate, Append, Rename as extensions).
- Master file vs Transaction file is a frequently confused pair — master = permanent data, transaction = temporary update data.

CHAPTER 1 SUMMARY

A file is a named, persistent collection of related records stored on secondary storage. It has attributes (name, identifier, type, location, size, protection, timestamps) and supports operations such as create, open, read, write, seek, close, delete, append, and rename. Files may be classified by content (text/binary), purpose (master/transaction/report), or organization (sequential/indexed/hashed). File systems offer permanence and simplicity but suffer from redundancy and limited integrity compared to databases.

CHAPTER 2: FILE ORGANIZATION AND ACCESSING TECHNIQUES

File organization refers to the way records are physically arranged/stored on a storage device, and the accessing technique refers to the method used to retrieve those records. The choice of organization affects storage efficiency, access speed, and ease of update. This chapter covers three major techniques: **Indexed**, **Sequential (Line Sequential)**, and **Hashed** file organization.

2.A INDEXED FILE ORGANIZATION

2.A.1 Introduction

Indexed file organization maintains the data records (in a **data file**) along with a separate auxiliary table called an **index**, which stores key values along with pointers (addresses) to the location of the corresponding record. This allows both sequential processing and direct (random) access to individual records without scanning the entire file.

2.A.2 Structure

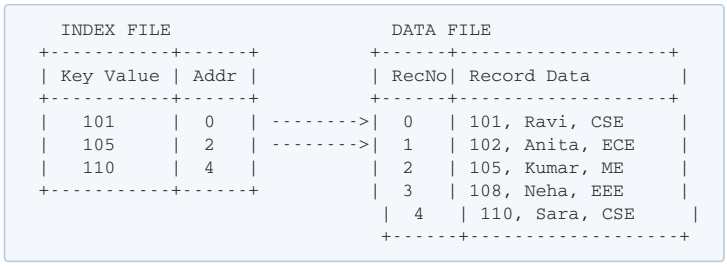


Fig 2.1 — Index file points to record locations in the data file

2.A.3 Working Mechanism

1. Records in the data file may be stored in any convenient order (often sorted by key for efficiency).
2. An index entry is created for selected key values, each containing the key and the disk address (block/record number) of that record.
3. To retrieve a record: the index is searched (often using binary search since it is small and sorted) to find the key, and the associated pointer is used to directly access the record in the data file.
4. Multiple indexes can exist on the same data file, allowing access by different keys (e.g., by Roll No. and by Name).

2.A.4 Types of Indexes

(a) Primary Index

An index built on the **primary key** field of the data file, which uniquely identifies each record. Since the primary key is unique, the index is also unique — each index entry corresponds to exactly one data record.

(b) Secondary Index

An index built on a **non-primary (candidate or non-key) field**, used to provide alternate access paths. A data file can have multiple secondary indexes (e.g., index on "Department" in an employee file in addition to the primary index on "Employee ID"). Secondary indexes are often **dense** because the field may not have a sorted physical ordering.

(c) Dense Index

An index that contains **one index entry for every record** in the data file. Every search key value in the data file appears in the index, along with a pointer to its record.

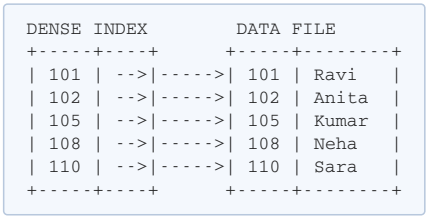


Fig 2.2 — Dense index: one entry per record

(d) Sparse Index

An index that contains entries **only for some records** — typically one entry per data block, pointing to the first record of that block. To find a record, the index is searched for the largest key value less than or equal to the search key, then the data block is scanned sequentially from that point.

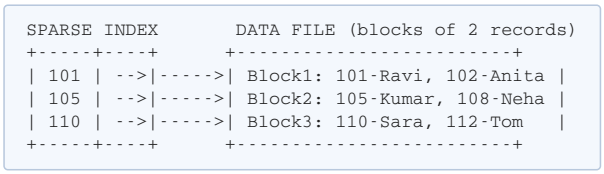


Fig 2.3 — Sparse index: one entry per block

Dense Index	Sparse Index
One index entry per record	One index entry per data block
Larger index size	Smaller index size
Faster lookup (direct pointer to record)	Slightly slower lookup (needs a block scan)
Higher memory usage	Lower memory usage
Used for primary or secondary indexes	Used for primary index on sorted files only

2.A.5 Advantages of Indexed File Organization

- Provides fast direct access to any record via its key, without scanning the whole file.
- Supports both sequential processing (by traversing the index) and random access.
- Multiple indexes allow access via different key fields.
- Efficient for files where records are frequently searched/retrieved individually.

2.A.6 Disadvantages of Indexed File Organization

- Requires extra storage space for maintaining the index(es).
- Index must be updated whenever records are inserted/deleted, adding overhead.
- More complex to implement compared to plain sequential files.
- If the index itself becomes large, multi-level indexing is required, increasing complexity.

2.A.7 Applications

- Banking systems — quick retrieval of account records by account number.
- Library catalog systems — lookup by book ID, author, or title (multiple indexes).
- Airline reservation systems — retrieval of bookings by PNR number.
- Student record systems — lookup by roll number or registration number.

2.A.8 Detailed Example

> **Worked Example 2.1**
>
> A college maintains a "Student" data file with 10,000 records sorted by Roll Number, stored in blocks of 50 records each (200 blocks total).
>
> **Sparse Index:** Contains 200 entries (one per block) — the roll number of the first record in each block, with a pointer to that block.
>
> **Search for Roll No. 4567:** Binary search the 200-entry sparse index to find the block whose starting roll number is ≤ 4567 and the next block's start is > 4567. Then sequentially scan that block (max 50 records) to find the exact record.
>
> **Dense Index:** Would contain all 10,000 entries, allowing a binary search directly to the exact record's pointer — faster lookup but 50× the index size of the sparse index.

> **EXAM-ORIENTED NOTES — Indexed Files**
>
> - Primary index → built on primary key, always unique.
> - Secondary index → built on non-key fields, provides alternate access paths, usually dense.
> - Dense = one entry per record; Sparse = one entry per block.
> - Sparse index works only if data file is sorted on the indexed field.
> - Trade-off: Dense index = faster but bigger; Sparse index = smaller but slightly slower.

2.B SEQUENTIAL (LINE SEQUENTIAL) FILE ORGANIZATION

2.B.1 Introduction

In sequential file organization, records are stored **one after another** in a physical/logical sequence, generally based on the order in which they are entered, or sorted by a key field. "Line sequential" refers specifically to text-based files where each record corresponds to one line, terminated by a line-feed/carriage-return character — a common format in COBOL and simple text-processing systems.

2.B.2 Structure

```
Record 1: 101, Ravi,  CSE,  85   \n
Record 2: 102, Anita, ECE,  90   \n
Record 3: 103, Kumar, ME,   78   \n
Record 4: 104, Neha,  EEE,  88   \n
              (stored contiguously, one record per line)
```

Fig 2.4 — Line sequential file: each record = one line

2.B.3 Storage Method

Records are written contiguously to storage in the order they are created or after sorting by a key field. Each record is delimited (in line-sequential files) by an end-of-line character so that programs can read record-by-record using simple line-reading functions.

2.B.4 Record Arrangement

Records may be arranged:

- **In entry order** — the order in which records were added (chronological).
- **In sorted order** — sorted by a key field (e.g., Roll No., Name) to support efficient sequential processing and merging with transaction files.

2.B.5 Access Process

To access a particular record, the system must start reading from the **beginning of the file** and read each record in turn, comparing it with the search key, until the desired record is found (or end-of-file is reached). Random access to the nth record requires reading through all $n-1$ preceding records.

```
Start --> [Rec1] --> [Rec2] --> [Rec3] --> ... --> [Rec n] --> EOF
          (must pass through every record to reach Rec n)
```

Fig 2.5 — Sequential access path

2.B.6 Advantages

- Simple to design, implement, and understand.
- Very efficient for batch processing where most/all records are processed in order (e.g., generating reports, payroll runs).
- Economical use of storage — no extra index space required.
- Easy to merge two sequential files that are sorted on the same key.

2.B.7 Disadvantages

- Very slow for random access — locating a single record may require scanning the entire file.
- Insertion of new records (to maintain sorted order) may require rewriting the whole file or maintaining a separate unsorted transaction area.
- Deletion typically requires marking records and periodic reorganization (compaction).
- Not suitable for applications requiring frequent, immediate updates to individual records.

2.B.8 Applications

- Payroll processing — processing all employee records once a month in sequence.
- Billing systems — generating bills for all customers in sequence.
- Log files — system/application logs appended sequentially over time.
- Batch report generation — printing mailing lists, attendance sheets, etc.

2.B.9 Detailed Example

> **Worked Example 2.2**
>
> A telephone company stores 1,00,000 customer records in a line-sequential file sorted by customer ID, to generate monthly bills.
>
> **Batch processing (efficient use):** A "transaction file" of meter readings (also sorted by customer ID) is read alongside the master file in a single pass; matching customer ID records are processed together to compute bills — this is highly efficient because both files are read sequentially exactly once.
>
> **Random lookup (inefficient use):** If a customer calls asking for their current balance, the system may need to scan, on average, 50,000 records (half the file) to find their record — illustrating why sequential files are unsuitable for interactive/random queries.

> **EXAM-ORIENTED NOTES — Sequential Files**
>
> - "Line sequential" = each record stored as one line of text, ending with a newline character.

- Best suited for batch processing; worst for random/direct access.
- Average search length for a sequential search = (n+1)/2 comparisons.
- Updating sorted sequential files often requires merging with a sorted transaction file (covered in Chapter 3).

2.C HASHED FILE ORGANIZATION

2.C.1 Introduction

Hashed (or direct/random) file organization uses a mathematical function, called a **hash function**, to compute the storage address of a record directly from its key value. This allows for very fast (close to O(1)) access to records without searching, since the address is calculated rather than looked up.

2.C.2 Hashing Concept

The key value of a record is passed through a hash function, which returns an address (or bucket number) where the record is stored. The same function is used both to **store** a record and to **retrieve** it — given the key, the address is recomputed and the record is fetched directly.

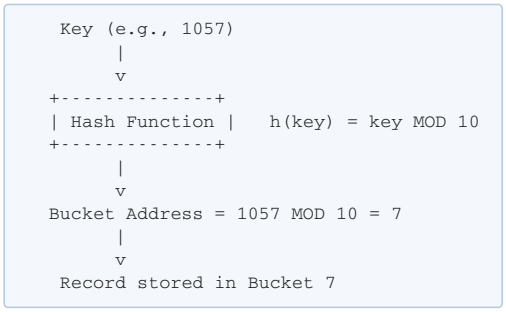


Fig 2.6 — Hashing process

2.C.3 Hash Functions

A good hash function should distribute records as **uniformly** as possible across the available address space to minimize collisions. Common hash functions include:

Method	Description	Example
Division (Modulo)	address = key MOD table_size	key=1057, size=10 → address = 7
Mid-Square	Square the key and extract middle digits	key=53 → 53²=2809 → mid digits "80"
Folding	Split key into parts and add them	key=123456 → 123+456=579
Truncation/Digit selection	Select specific digits from the key	key=983456 → take last 2 digits = 56

2.C.4 Bucket Organization

The hash address typically refers to a **bucket** — a storage unit (e.g., a disk block) that can hold multiple records. Using buckets reduces the frequency of collisions because several records with different keys but the same hash address can still fit in the same bucket, as long as the bucket is not full.

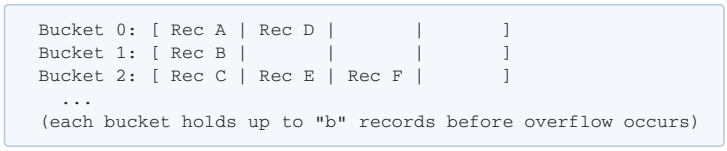


Fig 2.7 — Bucket-based hashed storage

2.C.5 Collision Handling Techniques

A **collision** occurs when two different keys hash to the same address/bucket and that bucket is already full. Techniques to handle collisions:

Technique	Description
Open Addressing (Linear Probing)	Search sequentially for the next free slot/bucket from the collision point
Quadratic Probing	Search for free slots at increasing quadratic intervals (1, 4, 9, ...) from the collision point
Double Hashing	Use a second hash function to compute the step size for probing
Chaining (Linked Lists)	Maintain a linked list of overflow records attached to each bucket

Rehashing	Apply a different hash function to find an alternate address on collision

2.C.6 Overflow Handling

Overflow occurs when a bucket is completely full and a new record hashes to it. Overflow handling techniques include:

- **Progressive Overflow:** The record is placed in the next available bucket in the file (similar to linear probing at the bucket level).
- **Linked Overflow (Chaining):** Each bucket has a pointer to a separate "overflow area"; overflow records are linked in a chain from the home bucket.
- **Re-hashing with a larger table:** When overflow becomes too frequent, the entire file may be reorganized into a larger hash table with a new hash function (dynamic/extendible hashing concept).

2.C.7 Advantages of Hashed File Organization

- Extremely fast direct access to records — ideally O(1), independent of file size.
- No separate index structure is required, saving storage space.
- Efficient for applications requiring fast lookups by a single key (e.g., real-time transaction systems).

2.C.8 Disadvantages of Hashed File Organization

- Records cannot be efficiently retrieved in sorted/sequential order by key.
- Collisions and overflows degrade performance and require careful handling.
- Choosing a good hash function that distributes keys uniformly is difficult.
- Range queries (e.g., "all records between key 100 and 200") are inefficient.
- File resizing (when it grows too full) can be expensive, requiring rehashing of all records.

2.C.9 Applications

- Real-time reservation systems requiring instant record lookup by ID.
- Symbol tables in compilers, mapping variable names to memory locations.
- Caches and in-memory key-value stores.
- Password verification systems (hashing usernames/keys for quick lookup).

2.C.10 Detailed Example

Worked Example 2.3

A hashed file has 10 buckets (numbered 0–9), each holding a maximum of 2 records. Hash function: h(key) = key MOD 10.

Insert keys in order: 23, 45, 13, 56, 33, 14.

Key	h(key)	Bucket	Action
23	23 mod 10 = 3	Bucket 3	Placed (Bucket 3: [23])
45	45 mod 10 = 5	Bucket 5	Placed (Bucket 5: [45])
13	13 mod 10 = 3	Bucket 3	Placed (Bucket 3: [23, 13]) — now full
56	56 mod 10 = 6	Bucket 6	Placed (Bucket 6: [56])
33	33 mod 10 = 3	Bucket 3 → FULL (collision)	Overflow: placed in linked overflow area of Bucket 3
14	14 mod 10 = 4	Bucket 4	Placed (Bucket 4: [14])

Retrieval of key 33: Compute h(33)=3, check Bucket 3 — not found among [23,13], follow overflow pointer — found in overflow area.

EXAM-ORIENTED NOTES — Hashed Files

- Hashing gives direct address computation — no searching required in ideal case.
- Collision = two keys map to same address; Overflow = bucket is full when a new record arrives.
- Common collision methods: linear probing, quadratic probing, double hashing, chaining.
- Hashed files are poor for range queries and sorted retrieval — key exam point for comparison questions.

2.D Comparison of the Three File Organization Techniques

Indexed	Sequential	Hashed
Access is via an index lookup, which gives the address of the record	Access is by a linear scan starting from the beginning of the file	Access is direct, computed using a hash function on the key
Random access speed is fast	Random access speed is slow	Random access speed is very fast
Sequential access is possible and efficient	Sequential access is the most efficient of the three	Sequential access is not efficient since records are not stored in order
Extra storage is needed to maintain the index	No extra storage is needed	No extra storage is needed, but an overflow area may be required
Insertion/deletion involves moderate effort, since the index must be updated	Insertion/deletion is difficult, often requiring reorganization of the file	Insertion/deletion is easy, unless an overflow occurs
Range queries are handled well	Range queries are handled well	Range queries are handled poorly
Best suited for applications needing a mix of sequential and random access	Best suited for pure batch processing	Best suited for pure random/direct access

CHAPTER 2 SUMMARY

Indexed file organization uses an auxiliary index (primary/secondary, dense/sparse) to enable fast direct access while preserving sequential access; it trades extra index storage for speed. Sequential (line sequential) file organization stores records contiguously in entry or sorted order and is ideal for batch processing but poor for random access. Hashed file organization computes record addresses directly using a hash function over buckets, giving near-instant access but requiring careful collision and overflow handling and performing poorly for range/sorted retrieval. The choice among the three depends on the relative frequency of random vs sequential access required by the application.

CHAPTER 3: FILE HANDLING FUNCTIONS

File handling functions are the operations performed on file data to maintain, organize, and process it efficiently. This chapter covers four key functions: **Sorting**, **Merging**, **Indexing**, and **Updating**.

3.A SORTING

3.A.1 Concept of Sorting in Files

Sorting is the process of arranging the records of a file into a particular order — ascending or descending — based on the value of one or more **key fields** (e.g., sorting an employee file by Employee ID or by Name).

3.A.2 Need for Sorting

- Sorted files enable efficient sequential processing and merging.
- Many algorithms (binary search, sparse indexing) require sorted data as a precondition.
- Reports such as bills, mailing lists, and result sheets are usually required in sorted order.
- Sorting is a prerequisite for the merge step in many file-updating procedures.

3.A.3 Internal Sorting

Internal sorting is used when the entire file (or dataset) can fit into the main memory (RAM) of the computer at once. The data is loaded into memory, sorted using algorithms like bubble sort, insertion sort, quick sort, or merge sort, and then written back to storage.

Property	Description
Memory Requirement	Entire dataset fits in RAM
Common Algorithms	Quick Sort, Heap Sort, Insertion Sort, Bubble Sort
Speed	Fast, since no disk I/O during sorting
Suitable For	Small to medium-sized files

3.A.4 External Sorting

External sorting is used when the file is too large to fit into main memory and must reside on secondary storage during the sorting process. It typically works in two phases:

1. **Sort Phase:** The large file is divided into smaller chunks that fit into memory; each chunk is sorted internally and written back to disk as a sorted "run".
2. **Merge Phase:** The sorted runs are merged together (often pairwise, in passes) until a single fully sorted file remains.

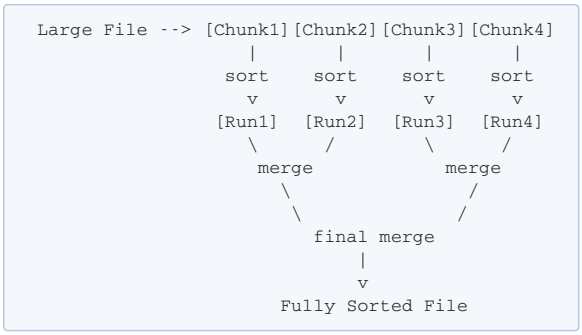


Fig 3.1 — External sort using sort & merge phases (e.g., merge sort)

Internal Sorting	External Sorting
Fits in RAM	Does not fit in RAM
Minimal/no disk I/O	Significant disk I/O (multiple passes)
Example: Quick Sort	Example: External Merge Sort
Used for small files	Used for large files (databases, big data)

3.A.5 Sorting Process (General Steps)

1. Read records from the input file (in chunks, if external).
2. Apply a comparison-based sorting algorithm on the key field.
3. If internal: write the sorted data back as the output file.

4. If external: write sorted chunks (runs) to temporary files, then repeatedly merge runs until one sorted file remains.

3.A.6 Advantages of Sorting

- Enables efficient searching (binary search) and indexing.
- Essential prerequisite for merging two or more files.
- Produces ordered reports required for business and administrative purposes.
- Improves the performance of sequential file processing.

3.A.7 Applications

- Generating alphabetically ordered student mark sheets.
- Arranging transaction records by date before processing.
- Preparing sorted master files for use with sparse indexes.
- Sorting large log files by timestamp for analysis.

3.A.8 Worked Example

> **Worked Example 3.1**
>
> A college has 50,000 student records stored on disk, but the computer's RAM can hold only 10,000 records at a time.
>
> **Step 1 (Sort Phase):** The file is divided into 5 chunks of 10,000 records each. Each chunk is loaded into RAM, sorted by Roll Number using Quick Sort, and written back as 5 sorted "runs" (Run1–Run5).
>
> **Step 2 (Merge Phase):** Run1 and Run2 are merged into a sorted file of 20,000 records (M1); Run3 and Run4 are merged into M2 (20,000 records); Run5 remains separate.
>
> **Step 3:** M1 and M2 are merged into a 40,000-record file, which is then merged with Run5 to give the final sorted file of 50,000 records.
>
> This is a classic example of **external merge sort**.

> **EXAM-ORIENTED NOTES — Sorting**
>
> - Internal sorting = data fits in RAM; External sorting = data exceeds RAM, needs disk-based runs.
> - External sort = Sort Phase (create sorted runs) + Merge Phase (combine runs).
> - Sorting is the foundation for sparse indexing, binary search, and merging operations.

3.B MERGING

3.B.1 Concept of Merging Files

Merging is the process of combining two or more **sorted** files into a single sorted file, while preserving the overall sorted order of the records based on the key field.

3.B.2 Need for Merging

- To combine a master file with a transaction file to produce an updated master file.
- To combine multiple sorted "runs" produced during external sorting into one fully sorted file.
- To consolidate data from multiple sources (e.g., branch-wise sorted records) into a single sorted file for centralized processing.

3.B.3 Types of Merging

Type	Description
Two-way Merge	Combines exactly two sorted files/runs into one sorted output
Multi-way (k-way) Merge	Combines k (more than two) sorted files/runs simultaneously into one sorted output
Balanced Merge	Distributes runs evenly across multiple tapes/files for efficient multi-pass merging

3.B.4 Merge Process

1. Open all input files (which must already be sorted on the same key) and place a "read pointer" at the start of each.
2. Compare the current records (pointed to) from each input file.
3. Write the record with the smallest (for ascending order) key value to the output file.
4. Advance the read pointer of the file from which the record was written.
5. Repeat steps 2–4 until all input files are exhausted.

```
File A (sorted): 10, 30, 50, 70
File B (sorted): 20, 40, 60, 80

Step-by-step two-way merge:
  compare 10,20 -> write 10  (A advances)
  compare 30,20 -> write 20  (B advances)
  compare 30,40 -> write 30  (A advances)
  compare 50,40 -> write 40  (B advances)
  compare 50,60 -> write 50  (A advances)
  compare 70,60 -> write 60  (B advances)
  compare 70,80 -> write 70  (A advances)
  A exhausted    -> write 80

Output: 10, 20, 30, 40, 50, 60, 70, 80
```

Fig 3.2 — Two-way merge of two sorted files

3.B.5 Advantages of Merging

- Efficient way to combine large sorted files without re-sorting from scratch.
- Each record from input files is examined only once (linear time complexity relative to total records).
- Forms the basis of external sort-merge algorithms for very large files.
- Allows incremental updates to master files using sorted transaction files.

3.B.6 Applications

- Updating a master file with a sorted transaction file (e.g., applying daily sales to inventory master).
- Merging sorted runs in external sorting algorithms.
- Combining sorted mailing lists from different departments into one master list.
- Consolidating sorted log files from multiple servers by timestamp.

3.B.7 Worked Example

> **Worked Example 3.2**
>
> Merge three sorted runs produced from external sorting:
>
> Run1: 5, 15, 25 | Run2: 10, 20, 30 | Run3: 12, 22, 28
>
> **3-way merge:** Compare the front elements of all three runs (5, 10, 12) → output 5 (Run1 advances to 15). Compare (15,10,12) → output 10 (Run2 advances to 20). Compare (15,20,12) → output 12 (Run3 advances to 22). Continue this process...
>
> Final merged output: 5, 10, 12, 15, 20, 22, 25, 28, 30
>
> This 3-way merge completes in a single pass, whereas pairwise two-way merging would require two passes (merge Run1&Run2, then merge result with Run3).

> **EXAM-ORIENTED NOTES — Merging**
>
> - Merging requires input files to be **already sorted** on the same key.
> - k-way merge reduces the number of passes required compared to repeated 2-way merges.
> - Merge is the core operation behind both external sorting and master-file updating.

3.C INDEXING

3.C.1 Concept of Indexing

Indexing, as a **file handling function**, is the process of creating and maintaining an auxiliary data structure (the index) that maps key values to the physical locations of records, in order to speed up data retrieval operations.

3.C.2 Purpose of Indexing

- To reduce the time required to locate a specific record, avoiding full-file scans.
- To support efficient implementation of indexed file organization (Section 2.A).
- To enable access to records via multiple keys (through multiple indexes).
- To support range queries and ordered retrieval efficiently.

3.C.3 Index Creation

Creating an index involves the following steps:

1. Decide on the key field(s) on which the index will be built (primary or secondary key).
2. Scan through the data file once, recording the key value and the corresponding address (block/record number) of each record (for a dense index), or of the first record of each block (for a sparse index).

3. Sort the resulting (key, address) pairs by key value, if not already in order.
4. Store this sorted list of pairs as the index file, separate from the data file.

3.C.4 Index Maintenance

Whenever the data file is updated, the index must also be updated to remain consistent:

Data File Operation	Required Index Maintenance
Insertion of a new record	Insert a new (key, address) entry into the index at the correct sorted position
Deletion of a record	Remove (or mark as deleted) the corresponding entry in the index
Modification of the key field	Remove the old index entry and insert a new entry for the updated key
Reorganization of the data file	Rebuild the index entirely to reflect new record addresses

3.C.5 Advantages of Indexing

- Drastically reduces the average search time for locating records (e.g., from O(n) to O(log n) using binary search on the index).
- Allows multiple access paths into the same data file via secondary indexes.
- Supports both random and sequential (via index traversal) access efficiently.

3.C.6 Applications

- Database management systems use indexing to speed up query processing.
- File systems use directory indexes to quickly locate files by name.
- Search engines use inverted indexes to map keywords to documents.
- Library catalog systems index books by title, author, and subject.

3.C.7 Worked Example

Worked Example 3.3

A data file with 8 records (block size = 2 records/block) has the following key values in order: 10, 15, 20, 25, 30, 35, 40, 45.

Creating a sparse primary index:

Block No.	Records in Block	Index Entry (First Key, Block Address)
Block 1	10, 15	(10, Block1)
Block 2	20, 25	(20, Block2)
Block 3	30, 35	(30, Block3)
Block 4	40, 45	(40, Block4)

Maintenance — inserting key 22: Record 22 belongs in Block 2 (between 20 and 25). If Block 2 has space, insert it there; the index entry (20, Block2) remains unchanged because 20 is still the first key of Block 2. If Block 2 is full, a split/overflow handling procedure is triggered and the index may need a new entry.

EXAM-ORIENTED NOTES — Indexing

- Index creation: scan data file → record (key, address) pairs → sort → store as index file.
- Index maintenance must handle insertion, deletion, modification, and reorganization.
- Indexing reduces search complexity significantly (linear → logarithmic).

3.D UPDATING

3.D.1 Concept of File Updating

Updating refers to the process of modifying the contents of a file to reflect new, changed, or removed information, keeping the file's data current and accurate. Updating typically involves three basic sub-operations: **insertion**, **deletion**, and **modification** of records.

3.D.2 Insertion of Records

Insertion adds a new record to the file.

File Organization	Insertion Procedure
Sequential (unsorted)	Simply append the new record at the end of the file
Sequential (sorted)	Insert in correct sorted position — may require shifting records or rewriting the file/using merge with a transaction file

Indexed	Insert the record into the data file and add a corresponding entry into the index
Hashed	Compute hash address and place record in the corresponding bucket; handle collision/overflow if bucket is full

3.D.3 Deletion of Records

Deletion removes an existing record from the file.

- **Logical deletion:** The record is marked with a "deleted" flag (tombstone) but remains physically present until the file is reorganized.
- **Physical deletion:** The record is actually removed and remaining records may be shifted/compacted to fill the gap.

For indexed and hashed files, the corresponding index entry or bucket entry must also be removed or marked.

3.D.4 Modification of Records

Modification (or amendment) changes the value of one or more fields in an existing record.

- If the modified field is **not the key**, the record can usually be updated in place.
- If the modified field **is the key** (in indexed/hashed files), this is effectively treated as a deletion of the old record plus an insertion of a new record, since the storage location/index entry depends on the key.

3.D.5 Update Procedures

Two general approaches are used for updating files:

Approach	Description	Best Suited For
Batch Update (using Transaction File)	All changes (insertions, deletions, modifications) are first recorded in a sorted "transaction file"; this is then merged with the sorted master file to produce a new updated master file	Sequential file organization, periodic processing
Direct (In-Place/Online) Update	Each change is applied immediately and directly to the master file (and its index/hash structure) as it occurs	Indexed and hashed file organization, real-time systems

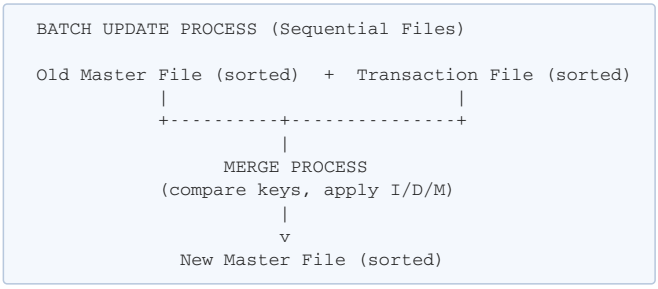


Fig 3.3 — Batch updating via merge of master and transaction files

3.D.6 Advantages of Proper Updating Procedures

- Keeps stored data accurate, current, and consistent with real-world events.
- Batch updating (merge-based) is efficient for processing large volumes of changes at once.
- Direct updating provides immediate consistency, essential for real-time applications.
- Maintains the integrity of associated structures (indexes, hash tables) when done correctly.

3.D.7 Challenges in Updating

- Maintaining sorted order in sequential files during insertion/deletion is costly.
- Index and hash structures must be kept synchronized with the data file after every update.
- Logical deletion (tombstones) can accumulate and waste space, requiring periodic reorganization/compaction.
- Concurrent updates by multiple users can cause conflicts without proper controls.
- Key-field modifications require special handling (delete + reinsert) in indexed/hashed files.

3.D.8 Worked Example

Worked Example 3.4

A sorted sequential master file of bank accounts (Account No., Name, Balance) is to be updated using a sorted transaction file containing three types of records: 'I' (Insert new account), 'D' (Delete account), 'M' (Modify balance).

Master File: 101-Ravi-5000, 102-Anita-7000, 105-Kumar-3000, 108-Neha-9000

Transaction File (sorted): (102, D), (103, I, Suresh, 4000), (105, M, +1000), (108, D)

Merge-update process (single pass, both files sorted by Account No.):

Compare	Action	Result Written to New Master

101 (master) vs 102 (txn)	101 has no transaction → copy as-is	101-Ravi-5000
102 (master) vs 102-D (txn)	Delete → do not copy 102	(skipped)
103-I (txn) vs 105 (master)	Insert new record 103 before 105	103-Suresh-4000
105 (master) vs 105-M (txn)	Modify balance: 3000+1000=4000	105-Kumar-4000
108 (master) vs 108-D (txn)	Delete → do not copy 108	(skipped)

New Master File: 101-Ravi-5000, 103-Suresh-4000, 105-Kumar-4000

EXAM-ORIENTED NOTES — Updating

- Three basic update operations: Insertion, Deletion, Modification.
- Logical deletion = mark as deleted (tombstone); Physical deletion = actually remove and compact.
- Batch (merge-based) updating suits sequential files; Direct/in-place updating suits indexed and hashed files.
- Key-field modification = delete old + insert new, for indexed/hashed files.

CHAPTER 3 SUMMARY

File handling functions maintain and process file data efficiently. Sorting arranges records by key — internal sorting when data fits in memory, external sorting (sort + merge phases) for large files. Merging combines sorted files into one ordered file and underlies both external sorting and master-file updating. Indexing creates and maintains auxiliary (key, address) structures to accelerate retrieval, requiring careful maintenance on every data change. Updating — comprising insertion, deletion, and modification — can be performed via batch processing (merging master and transaction files, suited to sequential organization) or direct in-place updates (suited to indexed and hashed organization), each presenting its own challenges for maintaining consistency.

END OF DOCUMENT

This guide covered: Concept of File (definition, characteristics, types, attributes, operations, advantages/limitations, examples); File Organization & Accessing Techniques (Indexed — primary/secondary, dense/sparse indexes; Sequential/Line Sequential; Hashed — hash functions, buckets, collisions, overflow); and File Handling Functions (Sorting — internal/external; Merging — two-way/k-way; Indexing — creation & maintenance; Updating — insertion, deletion, modification, batch vs direct). Use the exam-notes boxes and FAQ lists at the end of each section for quick revision.